arXiv:2409.12903v2 [cs.CL] 20 Sep 2024

Scaling Smart: Accelerating Large Language Model Pre-training with Small Model Initialization

Mohammad Samragh* **Iman Mirzadeh** **Keivan Alizadeh Vahid** **Fartash Faghri**

Minsik Cho **Moin Nabi** **Devang Naik** **Mehrdad Farajtabar***

Apple

Abstract

The pre-training phase of language models often begins with randomly initialized parameters. With the current trends in scaling models, training their large number of parameters can be extremely slow and costly. In contrast, small language models are less expensive to train, but they often cannot achieve the accuracy of large models. In this paper, we explore an intriguing idea to connect these two different regimes: Can we develop a method to initialize large language models using smaller pre-trained models? Will such initialization bring any benefits in terms of training time and final accuracy? In this paper, we introduce HyperCloning, a method that can expand the parameters of a pre-trained language model to those of a larger model with increased hidden dimensions. Our method ensures that the larger model retains the functionality of the smaller model. As a result, the larger model already inherits the predictive power and accuracy of the smaller model before the training starts. We demonstrate that training such an initialized model results in significant savings in terms of GPU hours required for pre-training large language models.

1 Introduction

Modern language models are very large, and training them is expensive [Kaplan et al., 2020, Rae et al., 2021, Hoffmann et al., 2022]. Experimenting with such models can be time-consuming and financially burdensome due to the high monetary cost. For instance, training a 12-billion-parameter model requires approximately 72,000 GPU hours [Biderman et al., 2023]. The total training cost from scratch can be expensive given current pricing of public cloud compute [Sevilla et al., 2022, Cottier et al., 2024]. Moreover, training can fail for reasons such as improper learning rate tuning, hardware failures, or loss divergence [Narayanan et al., 2021, Dubey et al., 2024]. Even with careful planning, robust engineering, and thorough testing to mitigate these failure risks, the monetary cost remains staggering.

While small language models are less costly to train and impose lower financial and environmental burdens during research and development, they often lack the desired level of accuracy. This situation leaves industries and businesses that prioritize performance with no choice but to scale up and utilize larger models. However, to address the prohibitive costs of training large language models from scratch, one effective strategy is to begin with a small language model and gradually expand its parameter capacity. This approach, known as model growth in contemporary literature, explores scaling up models from modest beginnings [Chen et al., 2015, Du et al., 2024].

*Correspondance to msamragh@apple.com and farajtabar@apple.com.

Preprint. Under review.

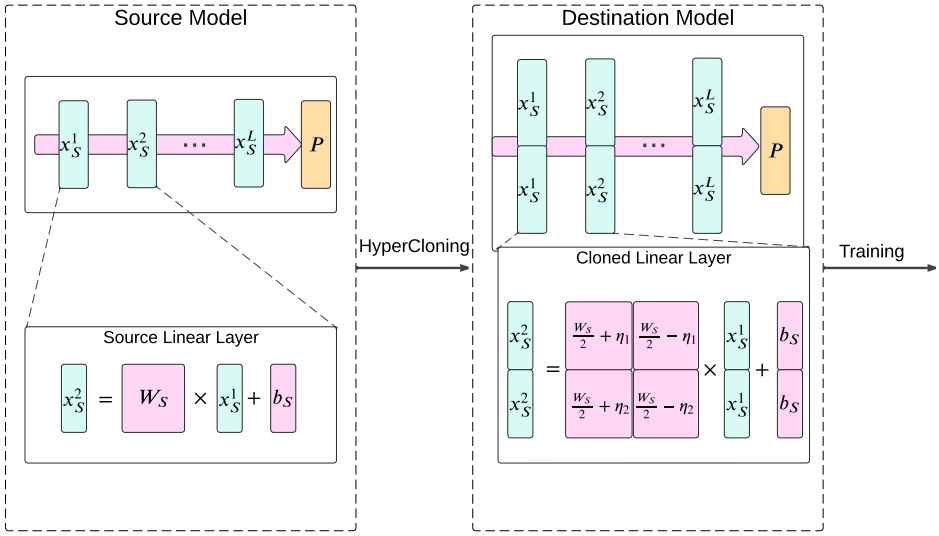
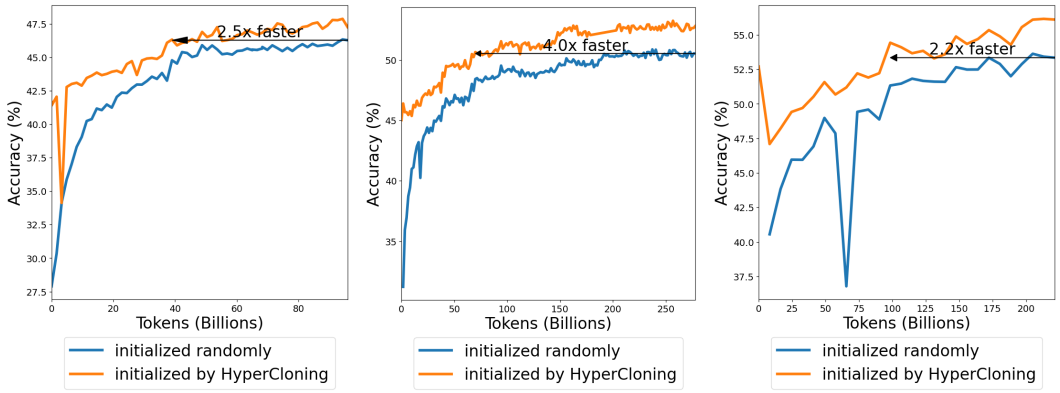


Figure 1: Illustration of HyperCloning. The parameters of the pretrained source network (left) are transferred to the destination network (right). In the destination model, both internal hidden representations and the final logits replicate those of the source network. This replication is achieved by precisely initializing the weights of the destination network's linear layers with the weights from the source network's linear layers, as depicted in the figure. Following this initialization, the destination network undergoes standard language model training. This initialization method enhances both the training speed and the final accuracy of the destination network.



(a) OPT (1.3B) (b) Pythia (1.4B) (c) OLMO (2.9B)

Figure 2: Benchmark accuracies (averaged over 10 tasks) when models are initialized with random weights and HyperCloning. Details are provided in the subsequent sections.

In this paper, we develop a method called HyperCloning to increase the hidden dimensions of transformer models, enabling the initialization of larger language models from smaller ones as depicted in Figure 1. Our method ensures a function-preserving transformation, where the output logits of the initialized model precisely match those of the smaller model. This functional preservation is advantageous as the larger language model achieves the same accuracy as the smaller model at the beginning of training. And further training enhances the accuracy of the large language model.

Our experiments show that HyperCloning enhances both training speed and final accuracy (given a finite and reasonable training budget) compared to the classic random initialization. We evaluate our method across three families of open-source language models, namely, OPT [Zhang et al., 2023], Pythia [Biderman et al., 2023] and OLMO [Groeneveld et al., 2024], summarizing the accuracy improvements and training speed gains in Figure 2.

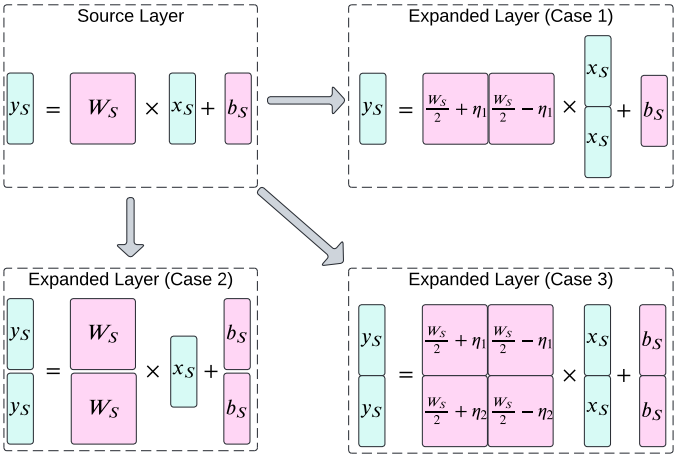


Figure 3: Demonstration of Linear layer cloning with 2-fold expansion, where W_s is the source model weight and η is a random noise matrix.

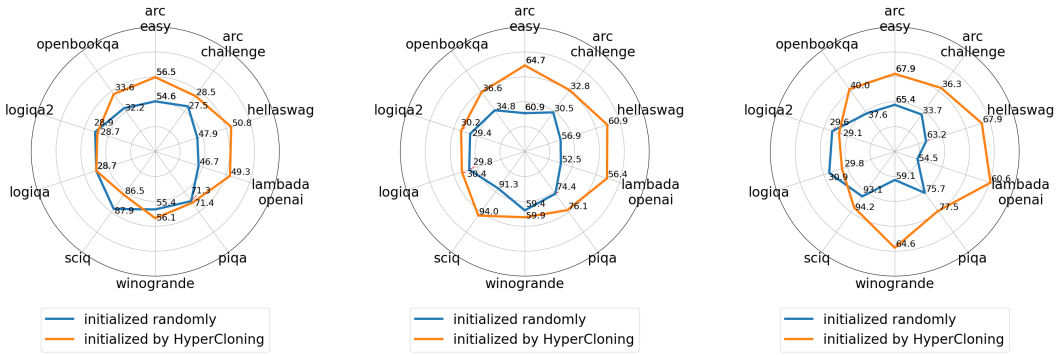
2 Methodology

Our goal is to design an oracle called HyperCloning that transfers the knowledge from a small pretrained language model to a larger model that requires training. To ensure the effectiveness of HyperCloning, we established several design goals:

- **Expansion Dimension:** The larger network should have larger hidden dimensions compared to the smaller network, while maintaining the same number of layers in both networks.
- **Function Preservation:** After converting the smaller model to its equivalent larger model, the logits in the final layers of both networks should match.
- **Low Compute Overhead:** The conversion process from the smaller model to the larger model should be straightforward, avoiding heavy computations or iterative updates.
- **Unchanged Training Loop:** For ease of deployment, the training loop should remain unchanged. The only modification should be in the network initialization.

In contrast to the mainstream model expansion approaches that increase the depth [Gong et al., 2019, Samragh et al., 2023, Yang et al., 2020, Karp et al., 2024, Li et al., 2023, Wang et al., 2023a], the first criteria targets a complementary techniques that can be accompanied by any of these methods to provide a full recipe for model scaling. Width scaling can be beneficial for increased model accuracy, robustness, and inference efficiency, compared to solely increasing depth. The second criterion gives the model a warm-start by ensuring that the larger model performs at least as well as the pretrained smaller model in the begining of training, leading to faster convergence and better final accuracy. As we'll see, our approach, also satisfies the third and fourth criteria, which are essential for maintaining efficiency and facilitating adoption in LLM training. These differentiate HyperCloning with expansions methods that use techniques such as distillation to transfer knowledge [Xu et al., 2024, Zhong et al., 2023], as they usually require changing the training setup.

Vector Cloning. Let $x_S \in \mathbb{R}^d$ be a hidden representation in the source (small) network. We achieve $x_D \in \mathbb{R}^{nd}$, the n-fold cloned version of x_S, by stacking n copies of x_S and denote it as $x_D = [\ x_S, \ldots, x_S\ ]^\top$. The main idea of HyperCloning is to establish the destination (large) network such that its hidden representations are cloned versions of the source (small) network. Consider a linear (fully connected) layer in the source network with weight parameter W_S and bias parameter b_S. The goal of HyperCloning is to obtain the weight W_D and bias b_D in the target network such that the input and output vectors in the target model are cloned versions of those in the source network. Depending on which of the input/output dimensions are expanded, there could be three different cases shown in Figure 3. Please refer to Appendix A for more specific details on the initializations for linear layers, attention layers, normalization layers, and positional embeddings.



(a) OPT (b) Pythia (c) OLMO

Figure 4: Benchmark accuracies over 10 tasks when models are initialized with random weights and HyperCloning.

3 Experiments

Model Architectures. We perform experiments with three open-source benchmarks: OPT [Zhang et al., 2023], Pythia [Biderman et al., 2023], and OMLO [Groeneveld et al., 2024]. We choose OPT-350M, Pythia-460M, and OLMO-1B as the base pretrained models. Using HyperCloning, we then construct three larger architectures as destination networks: OPT-1.3B, Pythia-1.4B, and OLMO-2.9B. Refer to Appendix B for more information about model architectures, training dataset, and training hyperparameters.

3.1 Results Overview

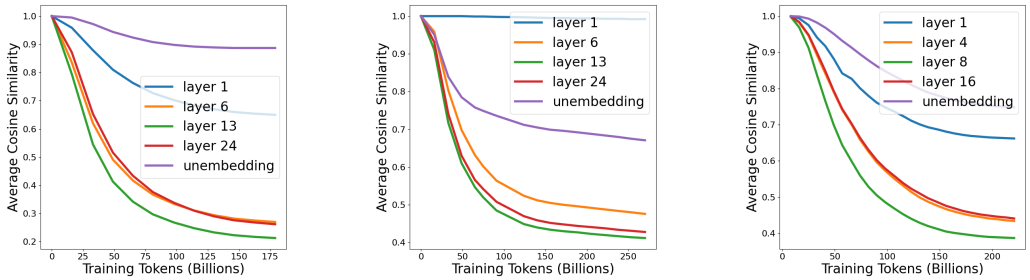
3.1.1 Comparison to Random Initialization

In this section, we compare the training convergence of the studied models in two scenarios: (i) random initialization, which is the standard process for training language models, and (ii) initialization with HyperCloning from a base model. In both cases, all other hyperparameters were kept identical, including learning rate, optimizer type, number of GPU nodes, batch size, context size, and order of training data.

We compute the models' accuracy using the Harness framework [Gao et al., 2023], an open-source and widely-used tool for LLM evaluation. Accuracies are measured over 10 different tasks, and the final accuracies for both random initialization and HyperCloning are presented in Figure 4. As shown, HyperCloning significantly improves the accuracy of the models after convergence.

Additionally, we measure the average accuracy over the 10 tasks and present its trend during training in Figure 2, which we present early in the paper. As observed, HyperCloning enables the network to reach the final accuracy of the random initialization baseline much faster, with a speedup ranging from 2.2x to 4x across different model types. The better final accuracy and faster convergence achieved by HyperCloning can be attributed to the transfer of knowledge from the base model. For example, the base model for the OLMO architecture was already trained on 2.4T tokens, and this knowledge was transferred to our model before training started. Note that the base models are freely available; we simply downloaded them from HuggingFace. In practice, HyperCloning can leverage previously trained models, thus offering a cost-saving advantage. Consequently, the model initialized with HyperCloning begins with high accuracy and can converge to a better solution with significantly fewer training tokens (i.e., 250B tokens rather than 2.4T).

One notable observation is that models initialized with HyperCloning tend to exhibit catastrophic forgetting at the beginning of training. This is evident in the training curve for the OLMO benchmark. However, our experiments show that with sufficient training, this forgetting can be compensated for. Despite the initial catastrophic forgetting, HyperCloning still outperforms random initialization by a large margin. Understanding the underlying causes of catastrophic forgetting, identifying strategies to mitigate it, and exploring why HyperCloning continues to outperform random initialization despite its occurrence are valuable avenues for future research. We believe these areas hold great potential for further enhancing our method.



(a) OPT (b) Pythia (c) OLMO

Figure 5: Evolution of average cosine similarities during training of the target network at up-project feed forward layers and the final unembedding layer.

3.2 Analyzing HyperCloning: Weight Symmetry

For an n-fold cloning, the target weights in the target network are initialized with blocks of source weights normalized by n. Consequently, the weights in the target network have a standard deviation that is $\frac{1}{n}$ of the standard deviation of the source network weights. This approach aligns with the standard deviation requirement proposed by [Glorot and Bengio, 2010] and offers benefits over existing methods like those in [Wang et al., 2023b] and [Chen et al., 2015].

However, our method, HyperCloning, initializes parts of the weight parameters as duplicates of each other. As noted by [Wang et al., 2023b], this duplication raises concerns that the duplicated neurons or weights might not learn independently, potentially limiting the model's capacity to utilize all parameters effectively. Nonetheless, we observe that this issue does not occur in our implementation, likely due to the randomness introduced by techniques such as dropout.

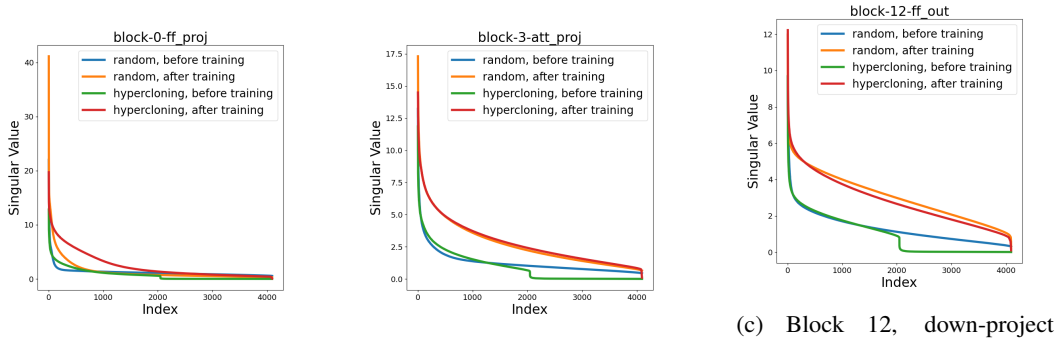
To analyze the evolution of these weight patterns during training, we define a metric to assess the symmetry in a cloned matrix. In the 2-fold cloning scenario depicted in Figure 3, case 3, each row of the matrix contains two identical horizontal vectors. We measure the cosine similarity between these vectors for each row and calculate the average cosine similarity across all rows. This metric provides an indication of the similarity between the vectors in the matrix.

Figure 5 shows the evolution of cosine similarities for several selected layers in our studied networks. Initially, the cosine similarities of all layers are 1, showing a complete symmetry in the weights. As training progresses, we observe that the cosine similarity decays in most layers. This suggests that the model is utilizing its effective parameter space during training. While this analysis provides insights into the evolution of model weights, further studies are worthwhile in the future.

3.3 Analyzing HyperCloning: Principal Components

Another way to analyze the convergence of HyperCloning is by examining the ranks of the weight matrices. Consider the weight matrices shown in Figure 3. Due to the replicating nature of our cloning algorithm, it is evident that the rank of the cloned matrix is at most equal to the rank of the base matrix. Essentially, the rank of the cloned matrix is half of its maximum possible value at initialization. This implies that, while the model has reasonable accuracy at initialization, it is not fully utilizing its capacity for making predictions. The concern is that the model might continue underutilizing this capacity even after training is completed. We demonstrate that this does not occur.

In Figure 6, we show the eigenvalues of several weight matrices within our OLMO-2.9B model before and after training, for both the randomly initialized model and the model initialized with HyperCloning. It can be seen that half of the singular values of the *before training* model initialized with HyperCloning are zero, whereas the randomly initialized model does not exhibit this behavior. However, after training, the model initialized with HyperCloning achieves similar high-rank weights to those in the randomly initialized model.



(a) Block 0, up-project weights

(b) Block 3, QKV weights

(c) Block 12, down-project weights

Figure 6: Singular values of weights at different layers of OLMO-2.9B model.

3.4 Alternative Expansion Methods

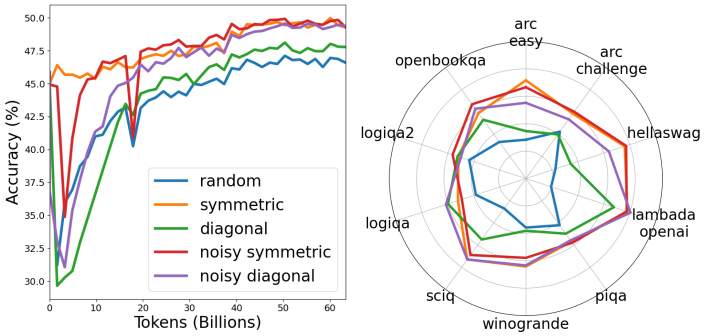
In our original formulation for the expanded weights, we proposed $W_L = \begin{bmatrix} \frac{W_S}{2} & \frac{W_S}{2} \\ \frac{W_S}{2} & \frac{W_S}{2} \end{bmatrix}$. However, this is not the only weight parameter configuration that can satisfy function preservation. In this part of our analysis, we empirically evaluate several strategies for initializing W_L as follows:

- **Symmetric**: $W_L = \begin{bmatrix} \frac{W_S}{2} & \frac{W_S}{2} \\ \frac{W_S}{2} & \frac{W_S}{2} \end{bmatrix}$.
- **Diagonal**: $W_L = \begin{bmatrix} W_S & 0 \\ 0 & W_S \end{bmatrix}$.
- **Noisy symmetric**: $W_L = \begin{bmatrix} \frac{W_S}{2} + \eta_1 & \frac{W_S}{2} - \eta_1 \\ \frac{W_S}{2} + \eta_2 & \frac{W_S}{2} - \eta_2 \end{bmatrix}$, where η_1 and η_2 are random noise tensors of the same shape as W_S.
- **Noisy diagonal**: $W_L = \begin{bmatrix} W_S + \eta_1 & -\eta_1 \\ \eta_2 & W_S - \eta_2 \end{bmatrix}$.

Note that all of the above weight expansion strategies are function-preserving. Figure 7 shows the accuracy of each instantiation method. The noise values (η_1 and η_2) in these experiments are selected such that the signal-to-noise ratio is 10 dB. All cloning methods outperform random initialization. The diagonal variant achieves the smallest accuracy boost, likely due to the presence of zero values in the expanded weight matrices. The noisy diagonal version performs slightly better than diagonal; however, the symmetric and noisy symmetric methods stand out as the best. With symmetric expansion, the benefits of noise addition are minimal. Therefore, we opt for the noise-free version of the method to avoid having to tune an extra hyper-parameter, the signal-to-noise ratio.

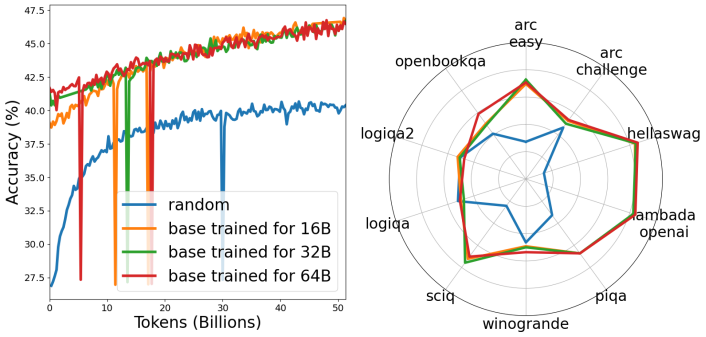
3.5 Effect of base model accuracy

Next, we study the effect of the base model's performance on the target model's performance. For this study, we use different checkpoints from the OPT-350M base model, trained with 16, 32, and 64 billion tokens, respectively. We initialize the target OPT-1.3B model with each of these checkpoints. Another baseline is random initialization, bringing the total number of comparison baselines to four. We observe the training convergence in Figure 8. As seen, initializing with the base model improves accuracy compared to random initialization when any of the base checkpoints are used for cloning. Among the cloned networks, those initialized with a more accurate base network achieve better accuracy, especially at the beginning of the training. However, as training continues, the differences between the curves become smaller.



(a) Average Accuracy. (b) Benchmark Accuracies.

Figure 7: Effect of expanding strategy on Model Accuracy for Pythia 1.4B training.

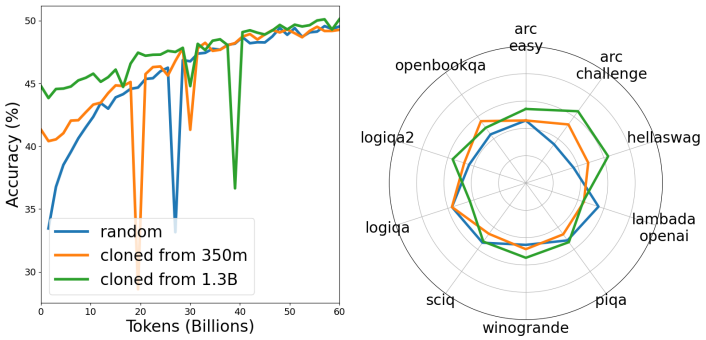


(a) Average Accuracy. (b) Benchmark Accuracies.

Figure 8: Effect of base model's accuracy on the convergence of target model. In this experiment, the base model is OPT-350M and the target model is OPT-1.3B.

3.6 Effect of base model size

Next, we demonstrate the effect of the base model's size on the target network's convergence. We create the target model by doubling the hidden dimension size of OPT-1.3B, resulting in a model we call OPT-5.3B. This architecture can be initialized in two ways using HyperCloning: (i) with OPT-1.3B using 2-fold cloning, or (ii) with OPT-350M using 4-fold cloning. The convergence of these candidates, along with the network initialized randomly, is shown in Figure 9. As observed, initializing with either OPT-350M or OPT-1.3B achieves faster convergence compared to random initialization, with OPT-1.3B providing better convergence than OPT-350M. This is because OPT-1.3B is larger and more accurate than OPT-350M, thereby offering a superior initialization.



(a) Average Accuracy. (b) Benchmark Accuracies.

Figure 9: Effect of base model's size on the convergence of target model. In this experiment, the base model is either OPT-350M or OPT-1.3B, and the target model is OPT-5.3B.

4 Related Work

A comprehensive study on related work in the network growth literature is available in [Du et al., 2024], which examines various growth strategies, including depth and width expansion. Their innovative approach to depth growth involves initializing a larger model by repeating block weights. This approach is also supported by the findings of other research work [Gong et al., 2019, Samragh et al., 2023, Yang et al., 2020, Karp et al., 2024, Li et al., 2023, Wang et al., 2023a]. For instance, [Samragh et al., 2023] demonstrates that, due to the presence of residuals in transformer architectures, blocks can be removed or duplicated to achieve superior initialization compared to random methods. In terms of width growth, [Du et al., 2024] explored several strategies, including directly copying weights, projecting weights to a larger dimension, initializing new weights to zero, and randomly initializing weights. Notably, both depth and width scaling strategies in their study do not preserve function properties. They concluded that depth growth achieves the best accuracies, while non-function-preserving width growth results in poorer performance. While their work provides valuable insights, our research takes a different direction by focusing on a function-preserving transformation in the width dimension. Further studies are necessary to fully understand the benefits of depth versus width scaling and function-preserving versus non-function-preserving transformations.

Width expansion was initially introduced by [Chen et al., 2015] for convolutional neural networks and later explored for BERT-style transformer models in [Chen et al., 2021]. Our work builds on these foundations by generalizing width expansion techniques to decoder-style transformers, which are increasingly utilized in modern large language models. Specifically, we extend the width expansion method to include attention layers, define essential cloning functions for position embeddings, and validate our approach through experiments on larger-scale models and datasets. These contributions advance the applicability of width expansion in contemporary transformer architectures.

In [Shen et al., 2022], the authors introduce a width expansion technique where the non-diagonal elements of the expanded weight matrices are initialized to zero. Our ablation studies indicate that this diagonal initialization can lead to slower convergence compared to our symmetric initialization method. In contrast, [Wang et al., 2023b] discuss that the symmetry of neurons in an expanded network suggests these neurons may not contribute independently to the model's learning. However, our experiments demonstrate that the symmetry in weights naturally breaks during training, potentially due to random operations such as dropout.

We further propose a function-preserving noise addition mechanism to intentionally break the symmetry in weights. Our findings show that this noise addition improves the model's convergence rate. Additionally, we analyze the eigenvalues of the expanded network's weights after training and find that their distribution closely resembles that of a network trained from scratch. This result suggests that the expanded network effectively utilizes its parameter space during learning, comparable to a network trained from scratch.

5 Conclusion

This paper introduces HyperCloning, a novel initialization strategy designed to transfer weights from a smaller, pretrained source model to a larger target model. The transfer process in HyperCloning is straightforward, effective, and preserves the model's functionality. By using this method, we achieve faster convergence and better final accuracy during language model training. In our experiments, HyperCloning accelerates training by 2-4 times. Additionally, we conducted ablation studies to explore the impact of the source model's architecture and different weight-cloning techniques on the target model's convergence.

References

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

Jack W Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, John Aslanides, Sarah Henderson, Roman Ring, Susannah Young, et al. Scaling language models: Methods, analysis & insights from training gopher. *arXiv preprint arXiv:2112.11446*, 2021.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pages 2397–2430. PMLR, 2023.

Jaime Sevilla, Lennart Heim, Anson Ho, Tamay Besiroglu, Marius Hobbhahn, and Pablo Villalobos. Compute trends across three eras of machine learning. In *2022 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2022.

Ben Cottier, Robi Rahman, Loredana Fattorini, Nestor Maslej, and David Owen. The rising costs of training frontier ai models. *arXiv preprint arXiv:2405.21015*, 2024.

Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15, 2021.

Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Tianqi Chen, Ian Goodfellow, and Jonathon Shlens. Net2net: Accelerating learning via knowledge transfer. *arXiv preprint arXiv:1511.05641*, 2015.

Wenyu Du, Tongxu Luo, Zihan Qiu, Zeyu Huang, Yikang Shen, Reynold Cheng, Yike Guo, and Jie Fu. Stacking your transformers: A closer look at model growth for efficient llm pre-training. *arXiv preprint arXiv:2405.15319*, 2024.

Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. Opt: Open pre-trained transformer language models, 2022. *URL https://arxiv. org/abs/2205.01068*, 3:19–0, 2023.

Dirk Groeneveld, Iz Beltagy, Pete Walsh, Akshita Bhagia, Rodney Kinney, Oyvind Tafjord, Ananya Harsh Jha, Hamish Ivison, Ian Magnusson, Yizhong Wang, et al. Olmo: Accelerating the science of language models. *arXiv preprint arXiv:2402.00838*, 2024.

Linyuan Gong, Di He, Zhuohan Li, Tao Qin, Liwei Wang, and Tieyan Liu. Efficient training of bert by progressively stacking. In *International conference on machine learning*, pages 2337–2346. PMLR, 2019.

Mohammad Samragh, Mehrdad Farajtabar, Sachin Mehta, Raviteja Vemulapalli, Fartash Faghri, Devang Naik, Oncel Tuzel, and Mohammad Rastegari. Weight subcloning: direct initialization of transformers using larger pretrained ones. *arXiv preprint arXiv:2312.09299*, 2023.

Cheng Yang, Shengnan Wang, Chao Yang, Yuechuan Li, Ru He, and Jingqiao Zhang. Progressively stacking 2.0: A multi-stage layerwise training method for bert training speedup. *arXiv preprint arXiv:2011.13635*, 2020.

Stefani Karp, Nikunj Saunshi, Sobhan Miryoosefi, Sashank J Reddi, and Sanjiv Kumar. Landscape-aware growing: The power of a little lag. *arXiv preprint arXiv:2406.02469*, 2024.

Xiang Li, Yiqun Yao, Xin Jiang, Xuezhi Fang, Xuying Meng, Siqi Fan, Peng Han, Jing Li, Li Du, Bowen Qin, et al. Flm-101b: An open llm and how to train it with 100 k budget. *arXiv preprint arXiv:2309.03852*, 2023.

Peihao Wang, Rameswar Panda, Lucas Torroba Hennigen, Philip Greengard, Leonid Karlinsky, Rogerio Feris, David Daniel Cox, Zhangyang Wang, and Yoon Kim. Learning to grow pretrained models for efficient transformer training. *arXiv preprint arXiv:2303.00980*, 2023a.

Xiaohan Xu, Ming Li, Chongyang Tao, Tao Shen, Reynold Cheng, Jinyang Li, Can Xu, Dacheng Tao, and Tianyi Zhou. A survey on knowledge distillation of large language models. *arXiv preprint arXiv:2402.13116*, 2024.

Ming Zhong, Chenxin An, Weizhu Chen, Jiawei Han, and Pengcheng He. Seeking neural nuggets: Knowledge transfer in large language models from a parametric perspective. *arXiv preprint arXiv:2310.11451*, 2023.

Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, 12 2023. URL `https://zenodo.org/records/10256836`.

Yite Wang, Jiahao Su, Hanlin Lu, Cong Xie, Tianyi Liu, Jianbo Yuan, Haibin Lin, Ruoyu Sun, and Hongxia Yang. Lemon: Lossless model expansion. *arXiv preprint arXiv:2310.07999*, 2023b.

Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 249–256. JMLR Workshop and Conference Proceedings, 2010.

Cheng Chen, Yichun Yin, Lifeng Shang, Xin Jiang, Yujia Qin, Fengyu Wang, Zhi Wang, Xiao Chen, Zhiyuan Liu, and Qun Liu. bert2bert: Towards reusable pretrained language models. *arXiv preprint arXiv:2110.07143*, 2021.

Sheng Shen, Pete Walsh, Kurt Keutzer, Jesse Dodge, Matthew Peters, and Iz Beltagy. Staged training for transformer language models. In *International Conference on Machine Learning*, pages 19893–19908. PMLR, 2022.

Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–16. IEEE, 2020.

A Cloning Details

In this section we explain the cloning process for different layer types in detail. For simplicity, we consider a 2-fold expansion of the network but the method can be generalized to a generalized n-fold expansion.

Cloning Linear Layers. In general, there can be three different expansion cases for a Linear Layer show in Figure 3:

- **Case 1:** Only the input is expanded: $x_D = \begin{bmatrix} x_S \\ x_S \end{bmatrix}$ and $y_D = y_S$. This may occur at any linear layer whose outputs are not expanded such as the unembedding layer.
- **Case 2:** Only the output is expanded: $x_D = x_S$ and $y_D = \begin{bmatrix} y_S \\ y_S \end{bmatrix}$. This may occur at any linear layer whose inputs are not expanded such as the embedding layer.
- **Case 3:** Both input and output are expanded : $x_D = \begin{bmatrix} x_S \\ x_S \end{bmatrix}$ and $y_D = \begin{bmatrix} y_S \\ y_S \end{bmatrix}$. This may occur at hidden linear layers which may include attention and/or feed-forward layers.

The expanded weight parameter is formed by stacking the original pretrained matrix in both rows and columns and normalizing the values by $\frac{1}{n}$, where n is the expansion factor in the input dimension. The expanded bias vector is created by repeating the original bias values n times. This formulation ensures that the outputs of the expanded linear layer are cloned versions of the original linear layer's outputs. More specifically:

- **Case 1:** We initialize $W_D = \begin{bmatrix} \frac{W_S}{2} + \eta_1 & \frac{W_S}{2} - \eta_1 \end{bmatrix}$ and $b_D = b_S$, where η_1 is a random tensor with reasonable magnitude. We then have:

$$y_D = W_D x_D + b_D = \begin{bmatrix} \frac{W_S}{2} + \eta_1 & \frac{W_S}{2} - \eta_1 \end{bmatrix} \begin{bmatrix} x_S \\ x_S \end{bmatrix} + b_S = y_S$$

- **Case 2:** We initialize $W_D = \begin{bmatrix} W_S \\ W_S \end{bmatrix}$ and $b_D = \begin{bmatrix} b_S \\ b_S \end{bmatrix}$. We then have:

$$y_D = W_D x_D + b_D = \begin{bmatrix} W_S \\ W_S \end{bmatrix} x_S + \begin{bmatrix} b_S \\ b_S \end{bmatrix} = \begin{bmatrix} W_S x_S + b_S \\ W_S x_S + b_S \end{bmatrix} = \begin{bmatrix} y_S \\ y_S \end{bmatrix}$$

- **Case 3:** We initialize $W_D = \begin{bmatrix} \frac{W_S}{2} + \eta_1 & \frac{W_S}{2} - \eta_1 \\ \frac{W_S}{2} + \eta_2 & \frac{W_S}{2} - \eta_2 \end{bmatrix}$ and $b_D = \begin{bmatrix} b_S \\ b_S \end{bmatrix}$, where η_1 and η_2 are a random tensors with reasonable magnitudes. We then have:

$$y_D = W_D x_D + b_D = \begin{bmatrix} \frac{W_S}{2} + \eta_1 & \frac{W_S}{2} - \eta_1 \\ \frac{W_S}{2} + \eta_2 & \frac{W_S}{2} - \eta_2 \end{bmatrix} \begin{bmatrix} x_S \\ x_S \end{bmatrix} + \begin{bmatrix} b_S \\ b_S \end{bmatrix} = \begin{bmatrix} y_S \\ y_S \end{bmatrix}$$

Cloning Attention Layers. When cloning attention layers, there are two possibilities to expand a multi-head attention:

- **Expanding the dimension of each attention head:** When increasing the head dimension, each of the query/key/value matrices can be treated as individual linear layers and expanded as explained in Figure 3. Let q_S and k_S represent the query and key values in the small network. Then the corresponding query and key values in the expanded network would be:

$$q_D = \begin{bmatrix} q_S \\ q_S \end{bmatrix} \quad \text{and} \quad k_D = \begin{bmatrix} k_S \\ k_S \end{bmatrix}$$

  The attention computed in the small network is:

$$a_S = \frac{q_S k_S^T}{\sqrt{d}}$$

  In the expanded layer, the attention is computed as:

$$a_D = \frac{q_D k_D^T}{\sqrt{2d}} = \frac{q_S k_S^T + q_S k_S^T}{\sqrt{2d}} = \sqrt{2} a_S$$

  To make a_D equal to a_S, we should scale the query value by $\frac{1}{\sqrt{2}}$. More generally, the expanded query weights should be scaled by $\sqrt{\frac{d_\text{S}}{d_\text{D}}}$, where d_S and d_D are the head dimensions in the original and extended layers, respectively.

- **Expanding the number of attention heads:** This case is straightforward. We can simply duplicate the attention heads.

In both cases, the fully connected layer that follows the attention layer will also be expanded to increase the hidden representation's dimensionality.

Cloning Layer Norm. let x_S be a hidden representation vector in the small network. Applying Layer Norm over this vector computes the following:

$$l(x_S) = \frac{x_S - \mathbb{E}(x_S)}{\sqrt{var(x_S) + \epsilon}} \cdot \gamma_S + \beta_S$$

When cloning layer norm, we expand the affine parameters (if any) as $\beta_D = \begin{bmatrix} \beta_S \\ \beta_S \end{bmatrix}$ and $\gamma_D = \begin{bmatrix} \gamma_S \\ \gamma_S \end{bmatrix}$. We then have:

$$l(x_D) = \frac{\begin{bmatrix} x_S \\ x_S \end{bmatrix} - \mathbb{E}(\begin{bmatrix} x_S \\ x_S \end{bmatrix})}{\sqrt{var(\begin{bmatrix} x_S \\ x_S \end{bmatrix}) + \epsilon}} \cdot \begin{bmatrix} \gamma_S \\ \gamma_S \end{bmatrix} + \begin{bmatrix} \beta_S \\ \beta_S \end{bmatrix} = \begin{bmatrix} l(x_S) \\ l(x_S) \end{bmatrix}$$

In the above derivation, we used the fact that $\mathbb{E}(\begin{bmatrix} x_S \\ x_S \end{bmatrix}) = \mathbb{E}(x_S)$ and $var(\begin{bmatrix} x_S \\ x_S \end{bmatrix}) = var(x_S)$. In general, repeating the weights and biases in the layer norm n-times will ensure that the output of the expanded layer norm is a cloned version of the output from the original layer norm. Similar argument is true for batch normalization, RMS normalization, and group normalization.

Cloning Positional Embedding Layers. For positional embedding, we need to define the n-times cloned equivalents. Let $P_S(x_S, i) \in \mathbb{R}^d$ denote the positional embedding of a pretrained small network. The n-times cloned positional embedding is defined as follows:

$$P_D(X_D, i) = \begin{bmatrix} P_S(x_S, i) \\ \vdots \\ P_S(x_S, i) \end{bmatrix}$$

In essence, the positional embedding of the expanded network is created by repeating the positional embedding of the small network n times. In our codebase, we define Pytorch equivalents of the expanded positional embedding layers when necessary.

B Architectures and Training Details

The architectures of our etudied networks are summarized in Table 1. Among the target models, OPT-1.3B and Pythia-1.4B are already available through HuggingFace, providing us with a good baseline for comparison. OLMO-2.9B was not trained by the authors of [Groeneveld et al., 2024], and we are the first to train and evaluate it. We obtain the weight checkpoints of the base models from the HuggingFace repositories, except for OPT-350M, for which we train our own base model with 30B tokens. This is because the HuggingFace OPT-350M model has extra linear layers after the embedding layer and before the unembedding layer, which the target OPT-1.3B model does not have. With these benchmarks, we emulate three different scenarios:

- **OPT**: The training dataset is the same for both the base and target model. The base model is trained with a relatively small number of tokens (30B).
- **Pythia**: The dataset used for training the base model (Pile) is not available to train the target model. We use a different dataset (DOLMA) for training the target model. The base model was trained with a moderate number of tokens ($\tilde{2}$50B).
- **OLMO**: The training dataset is the same for both the base and target model. The base model is trained with a large number of tokens (2.4T).

Table 1: Summary of base and target model architectures.

	Model	#L	#H	d_{model}	d_{FFN}
base	OPT 350M	24	16	1024	4098
target	OPT 1.3B	24	32	2048	8192
base	Pythia 410M	24	16	1024	4098
target	Pythia 1.4B	24	32	2048	8192
base	OLMO 1B	16	16	2048	16384
target	OLMO 2.9B	16	32	4096	16384

Dataset. For all experiments, we use the DOLMA dataset provided by the authors of [Groeneveld et al., 2024]. This dataset includes several open-source datasets and totals up to 2.4 trillion tokens. However, our training jobs do not process this many tokens due to the extensive cost. To ensure fair representation of all sub-datasets within DOLMA, we shuffled the data shards. The seed for random shuffling is kept the same across all our experiments to eliminate the impact of data ordering on our conclusions.

Training Parameters. For all of our experiments, we use the AdamW optimizer with a weight decay of 0.05, $\beta_1 = 0.9$ and $\beta_2 = 0.999$. We use gradient accumulation with 16 steps to increase our effective batch size and the zero_2 gradient update algorithm [Rajbhandari et al., 2020] to reduce

memory footpring. We apply a learning rate warm-up over 25,000 iterations to reach the maximum learning rate. Afterward, we decay the learning rate to 1/10th of its value until 2,500,000 iterations, after which the learning rate is kept constant. Our models are trained on 64 GPUs with varying batch sizes, context sizes, and learning rates summarized in Table 2.

Table 2: Training hyperparameters.

Model	Batch Size	Context Size	Max LR	Average Tokens Per Iteration
OPT 1.3B	2	1024	1.5E-4	65K
Pythia 1.4B	2	1024	1.5E-4	65K
OLMO 2.9B	2	2048	3E-4	82K